Trabalho Prático – Privacidade Diferencial em Dados Não-IID no Flautim

João Gabriel Vaz (jgvaz@ufmg.br)
Giordano Henrique Liporati (ghl2023@ufmg.br)
Giovanni Russo Paschoal (grpaschoal@ufmg.br)
Leonardo Romano (leo.andrade2001@gmail.com)
Departamento de Ciência da Computação da UFMG (DCC/UFMG)
Belo Horizonte, Minas Gerais - Brasil

2026/1

1 Introdução

O trabalho proposto trata-se de um experimento que compara algumas das diferentes técnicas de Privacidade Diferencial baseadas em Gradient Clipping, e avalia seus impactos sobre distribuições de dados não-Independente e Identicamente Distribuídas (dados Não-IID). Para isso, foram realizados experimentos que utilizam o framework *Flower*, para treinamento de sistemas de Aprendizado Federado, e da ferramenta do *Flautim*, que é disponibilizada pelo laboratório *FutureLab*, da UFMG.

2 Aprendizado Federado

O Aprendizado Federado (Federated Learning – FL) é um paradigma de aprendizado de máquina descentralizado que permite o treinamento de modelos em múltiplos dispositivos locais (clientes) sem a necessidade de compartilhar os dados brutos com um servidor central. Desenvolvido para lidar com a crescente preocupação em relação à privacidade e segurança de dados, o FL inverte a lógica tradicional, pois, no lugar de levar os dados até o modelo, leva-se o modelo até os dados.

O processo geralmente ocorre em rodadas de comunicação. Inicialmente, um servidor central distribui um modelo global (ou seus pesos iniciais) para uma amostra de clientes. Cada cliente realiza o treinamento localmente utilizando seu próprio conjunto de dados privados, calculando os gradientes e atualizando os pesos do modelo. Em seguida, os clientes enviam apenas essas atualizações do modelo de volta para o servidor. O servidor, por sua vez, agrega essas atualizações, utilizando algoritmos como o *Federated Averaging* (FedAvg), para gerar uma nova versão melhorada do modelo global. Esse ciclo se repete até a convergência do modelo. Embora o Aprendizado Federado reduza significativamente os riscos de exposição dos dados, a transmissão dos gradientes ainda pode ser vulnerável a ataques de inferência, o que motiva a integração com técnicas de Privacidade Diferencial.

3 Privacidade Diferencial

A Privacidade Diferencial (PD) é um arcabouço matemático rigoroso que fornece fortes garantias estatísticas contra a identificação de indivíduos em um conjunto de dados. O objetivo da PD é garantir que a saída de um algoritmo não sofra alterações significativas caso um único registro seja adicionado ou removido da base de dados.

Matematicamente, um algoritmo randomizado $\mathcal{M}$ garante (ϵ, δ)-Privacidade Diferencial se, para quaisquer dois conjuntos de dados vizinhos D e D' (que diferem em no máximo um elemento) e para qualquer subconjunto de saídas possíveis $S \subseteq \text{Range}(\mathcal{M})$, a seguinte inequação for satisfeita:

$$\Pr[\mathcal{M}(D) \in S] \leq e^{\epsilon} \Pr[\mathcal{M}(D') \in S] + \delta$$

Onde ϵ (orçamento de privacidade) quantifica a variação máxima permitida na distribuição de saída, limitando o ganho de informação que um adversário pode obter; e δ representa a probabilidade de que a garantia de ϵ falhe. Valores menores de ϵ oferecem maior privacidade, porém geralmente reduzem a utilidade do modelo.

Nesse trabalho, será usada a privacidade diferencial aplicada ao contexto de Machine Learning através do algoritmo DP-SGD (Differentially Private Stochastic Gradient Descent). Para garantir a PD no treinamento, o algoritmo adiciona ruído estatístico (geralmente Gaussiano) às atualizações do modelo. No entanto, a quantidade de ruído necessária depende da "sensibilidade"da função — ou seja, o quanto uma única amostra pode alterar o resultado. Como os gradientes não possuem um limite natural máximo, utilizamos a técnica de *Gradient Clipping* para impor um limite máximo artificial à norma L_2 dos gradientes, permitindo assim o cálculo exato do ruído que deve ser adicionado para satisfazer a condição (ϵ, δ).

3.1 Gradient Clipping de Corte Fixo

O Gradient Clipping de Corte Fixo consiste em limitar a magnitude do gradiente computado para cada amostra de treinamento a um valor máximo pré-determinado, definido pela constante C. Se a norma L_2 do gradiente for menor ou igual a C, o gradiente permanece inalterado. Caso contrário, ele é escalonado para baixo para que sua norma seja exatamente C. Matematicamente, dado um gradiente computado g, o gradiente cortado $\tilde{g}$ é calculado como:

$$\tilde{g} = \frac{g}{\max\left(1, \frac{\|g\|_2}{C}\right)}$$

Embora eficaz, essa abordagem possui como principal desvantagem a necessidade de definir o hiperparâmetro C empiricamente. Se C for muito grande, a sensibilidade aumenta e muito ruído será adicionado, prejudicando o modelo. Se C for muito pequeno, a direção real do gradiente é fortemente distorcida pelo corte agressivo, retardando ou impedindo a convergência.

3.2 Gradient Clipping de Corte Adaptativo

Como a distribuição das normas dos gradientes varia significativamente durante as etapas de treinamento, o Gradient Clipping de Corte Adaptativo ajusta o valor do limite de corte C de forma dinâmica. O objetivo é adaptar C iterativamente para que uma proporção alvo dos gradientes não seja cortada, mitigando a necessidade de ajuste manual do hiperparâmetro.

Matematicamente, seja C_t o limite de corte no instante t e $\gamma \in [0, 1]$ a taxa alvo de quantil (por exemplo, $\gamma = 0.5$ indica que queremos que metade dos gradientes sofram o corte e a outra metade não). O algoritmo avalia se o gradiente sofreu o corte através de uma função indicadora $b = \mathbb{I}(\|g\|_2 \leq C_t)$, onde $b = 1$ se o gradiente está dentro do limite e $b = 0$ se exceder. A atualização do limite para o próximo passo é feita com base no desvio da proporção alvo:

$$C_{t+1} = C_t \cdot \exp\left(-\eta(b - \gamma)\right)$$

Onde η atua como uma taxa de aprendizado para a atualização do limiar de corte. Essa abordagem melhora a eficiência da inserção do ruído e ajuda a estabilizar o treinamento sob privacidade, especialmente em ambientes federados.

4 O framework Flower

O *Flower* é um framework open-source de Aprendizado Federado projetado para ser altamente modular, independente de *frameworks* de aprendizado profundo e escalável (2). Ele permite a criação de redes de aprendizado federado sem alterar o código base de treinamento nativo do PyTorch, TensorFlow ou Keras.

A arquitetura do Flower baseia-se em uma separação clara de responsabilidades entre um Servidor Central e múltiplos Clientes. O servidor é encarregado de orquestrar as rodadas de comunicação, selecionar os clientes disponíveis e agregar os modelos locais utilizando estratégias como o FedAvg (*Federated Averaging*). Os clientes, por sua vez, executam o treinamento localmente em seus próprios ambientes e enviam apenas os parâmetros atualizados de volta ao servidor. Essa flexibilidade viabiliza a execução de cenários complexos de federação de dados, além de suportar nativamente a integração com bibliotecas complementares de privacidade, como as abordagens de corte de gradiente e ruído estatístico mencionadas anteriormente.

5 Flautim

O *Flautim* é uma plataforma voltada à validação de modelos de aprendizado federado. (5) Trata-se de uma abordagem que permite o treinamento de modelos com dados distribuídos, sem a necessidade de compartilhamento de dados sensíveis, garantindo maior segurança e conformidade regulatória, além de permitir o treinamento de modelos de inteligência artificial diretamente nas fontes de dados, como veículos e sistemas distribuídos, sem a necessidade de centralização. A iniciativa faz parte da Linha VI – Conectividade Veicular, do programa Mover, com coordenação da Fundação de Apoio da UFMG (Fundep), A coordenação técnica é da Universidade Federal de Minas Gerais (UFMG), sob responsabilidade do *FutureLab* (Laboratório de Pesquisa e Inovação do DCC-UFMG).

6 O dataset FMNIST

O conjunto de dados utilizado foi o FMNIST (Fashion-MNIST) (6), uma vez que ele era um dos conjuntos disponíveis nos tutoriais de exemplo do *Flautim* (4). Trata-se de um banco de imagens de artigos de vestuário criado pela Zalando e disponibilizado como um substituto direto e mais desafiador para o clássico dataset MNIST. O conjunto original de dígitos escritos à mão tornou-se demasiadamente fácil para algoritmos modernos de visão computacional, o que motivou a adoção do Fashion-MNIST.

O dataset preserva a estrutura e o formato exato do MNIST original, compreendendo um total de 70.000 imagens em tons de cinza com resolução de 28×28 pixels. Desse total, 60.000 imagens formam o conjunto de treinamento e 10.000 o conjunto de testes. As imagens estão divididas igualmente em 10 classes distintas (0: Camiseta/Top, 1: Calça, 2: Suéter, 3: Vestido, 4: Casaco, 5: Sandália, 6: Camisa, 7: Tênis, 8: Bolsa, 9: Bota). No contexto de Aprendizado Federado, o FMNIST representa um passo intermediário, exigindo maior complexidade arquitetônica do que o reconhecimento de dígitos, demonstrando claramente a degradação de performance que partições não-IID e ruído de privacidade causam, mas sem as grandes sobrecargas computacionais associadas a bases como ImageNet ou CIFAR.

7 Dados não-IID

O conceito de dados não-IID (Não Independente e Identicamente Distribuídos) reflete a realidade das redes descentralizadas. Em ambientes reais de Aprendizado Federado, cada cliente coleta dados conforme o seu uso e contexto local. Consequentemente, a distribuição das classes dos atributos e da quantidade de dados disponíveis varia drasticamente de um cliente para outro. Essa falta de uniformidade e independência é um dos maiores desafios em FL, pois o treinamento local com dados muito tendenciosos pode fazer com que os pesos locais divirjam excessivamente, fenômeno conhecido como *weight divergence*. Isso dificulta a capacidade do servidor de agregar os modelos para formar um modelo global coerente.

7.1 Particionador de Dirichlet

Para simular o desbalanceamento não-IID, foi utilizado a Distribuição de Dirichlet. Este método gera partições contínuas de dados ao controlar estatisticamente o grau de heterogeneidade das classes entre os clientes.

Matematicamente, a proporção de instâncias da classe k que será alocada para cada um dos N clientes é obtida ao amostrar de uma distribuição de Dirichlet controlada por um hiperparâmetro de concentração $\alpha > 0$:

$$\boldsymbol{p}_k \sim \mathrm{Dir}(\alpha \mathbf{1}_N)$$

Onde $\mathbf{1}_N$ é um vetor de tamanho N preenchido com uns, e $\boldsymbol{p}_k = (p_{k,i})_{i=1}^{N} = (p_{k,1}, p_{k,2}, \ldots, p_{k,N})$ representa a probabilidade de a amostra da classe k ser designada ao cliente i. O parâmetro α dita o quão não-IID serão os dados. Quando $\alpha \to \infty$, a distribuição tende a ser uniforme, com comportamento IID. Em contrapartida, quando $\alpha \to 0$, o particionamento se torna altamente não-IID, com alguns clientes recebendo a vasta maioria dos exemplos de uma classe específica e nenhum de outras.

No framework *Flower*, através da biblioteca complementar `flower-datasets`, o *Dirichlet Partitioner* operacionaliza essa lógica. Ao instanciar o particionador, os rótulos do dataset são passados e é escolhido o valor de α, delegando à ferramenta a tarefa de fatiar o dataset de acordo com a distribuição gerada estatisticamente, mapeando as partições não-IID para as requisições de clientes no momento da simulação. A diferença no framework, é que um valor de α próximo de 1 implica em dados totalmente IID, enquanto um valor próximo de zero implica na maior heterogeneidade possível.

Foi então realizado um experimento simples para comparar como o classificador básico exemplificado no tutorial 4 do Flautim (3), sem Privacidade Diferencial, se sai em relação à heterogeneidade dos dados do FMNIST, para diferentes valores de α. O modelo é o *PoCModel_2HiddenLayers*, que é um MLP Profundo com duas camadas ocultas ($784 \to 256 \to 128 \to 10$), criado especialmente para o FMNIST, de acordo com o paper *Power of Choice* (1).

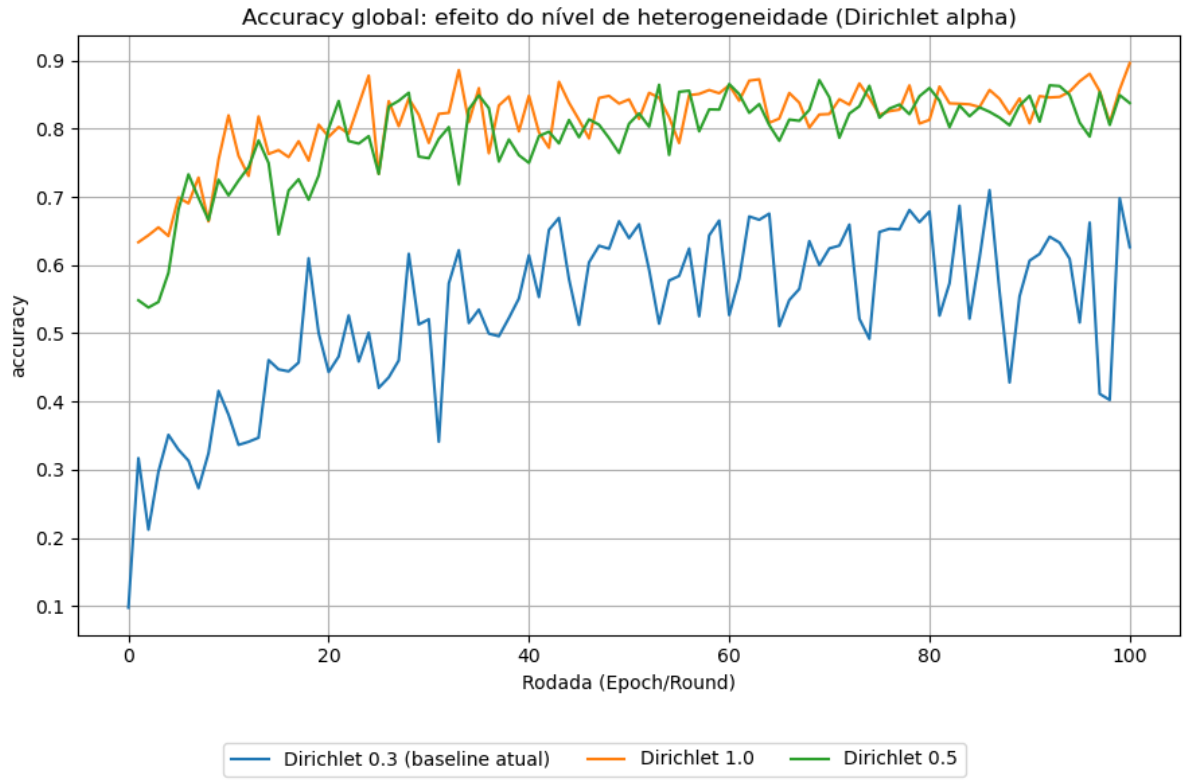


Figura 1: Impacto dos diferentes valores de α na acurácia do classificador

O experimento de $\alpha = 0.3$ foi utilizado como *baseline* para as outras comparações presentes nesse trabalho, uma vez que nosso objetivo foi de testar como a utilidade das técnicas de Privacidade Diferencial é afetada em distribuições de dados muito heterogêneas.

8 Experimentos

Nos nossos experimentos, realizamos o Server Side Gradient Clipping nas máquinas do Flautim, com agregação por *Federated Averaging* (FedAvg). Ou seja, foi assumido nos experimentos um servidor confiável, já que o objetivo principal foi testar o efeito dos dados não-IID nos modelos. Cada etapa de agregação foi realizada com 10 clientes por vez, de um total de 100 clientes disponíveis. Ao todo, foram realizadas 100 rodadas (épocas), e medidas a acurácia, F1 Score e Loss globais e para cada cliente, em cada rodada.

Além da comparação inicial para determinar qual seria nosso classificador *baseline*, realizamos o mesmo experimentos com Privacidade Diferencial, comparando, também para um $\alpha = 0.3$ duas versões com Gradient Clipping de Corte Fixo, uma com Clipping Norm $C = 1.0$ e outra com Clipping Norm $C = 5.0$, ambos com ruído Gaussiano com um orçamento de privacidade $\epsilon = 0.1$. Também testamos o Gradient Clipping de Corte Adaptativo, com o mesmo ϵ alvo e α. Por fim, realizamos um último teste de baseline, sem Privacidade Diferencial, com uma distribuição IID com $\alpha = 1.0$, apenas para fins comparativos de um cenário não diferencialmente privado considerado "ideal".

Os resultados dos experimentos realizados podem ser observados nos seguintes gráficos:

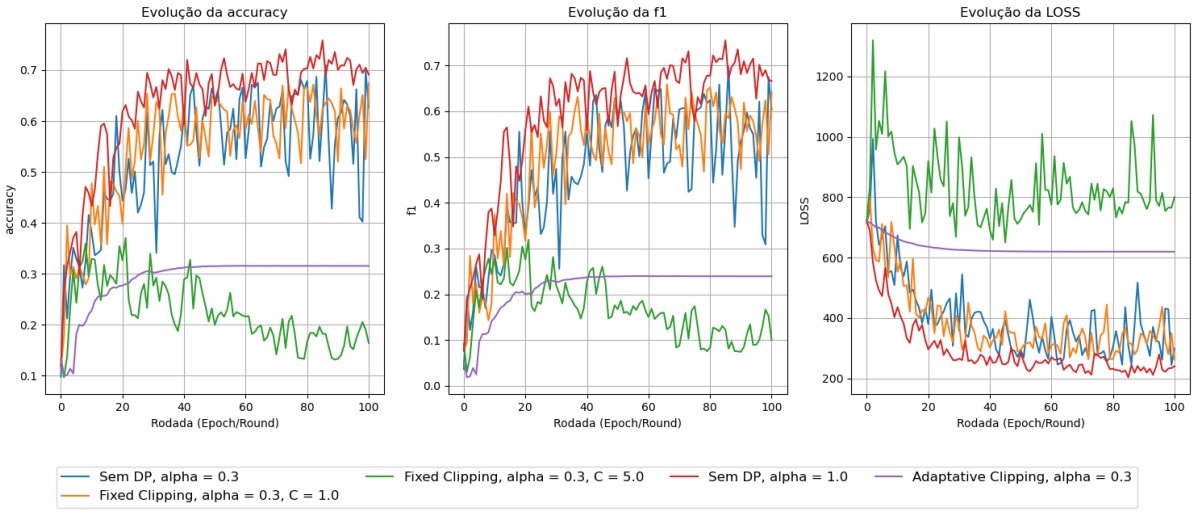


Figura 2: Comparativo entre a F1 score, acurácia e Loss das técnicas de DP

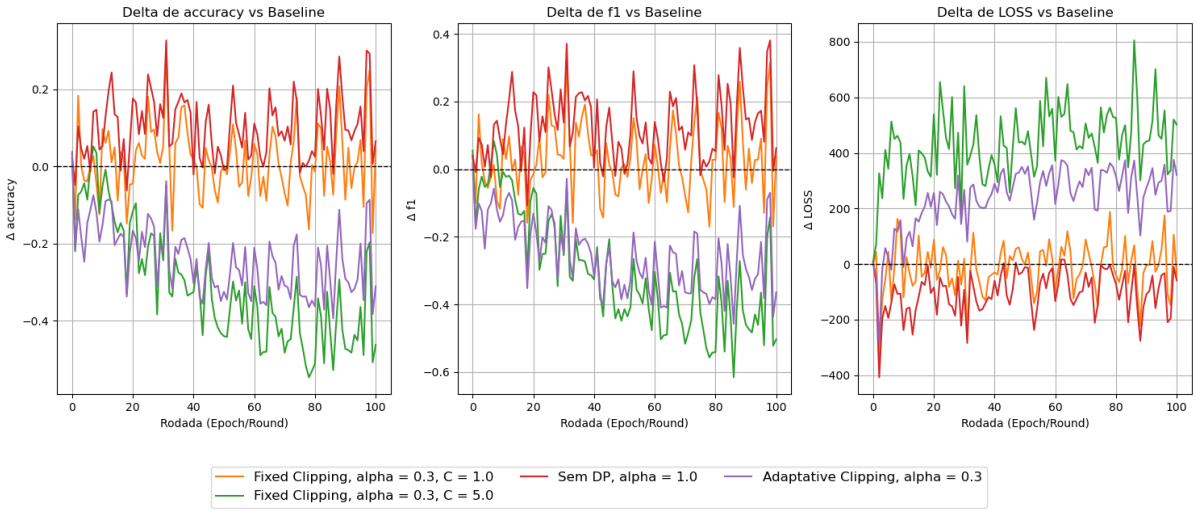


Figura 3: F1 score, acurácia e Loss das técnicas de DP em relação ao baseline

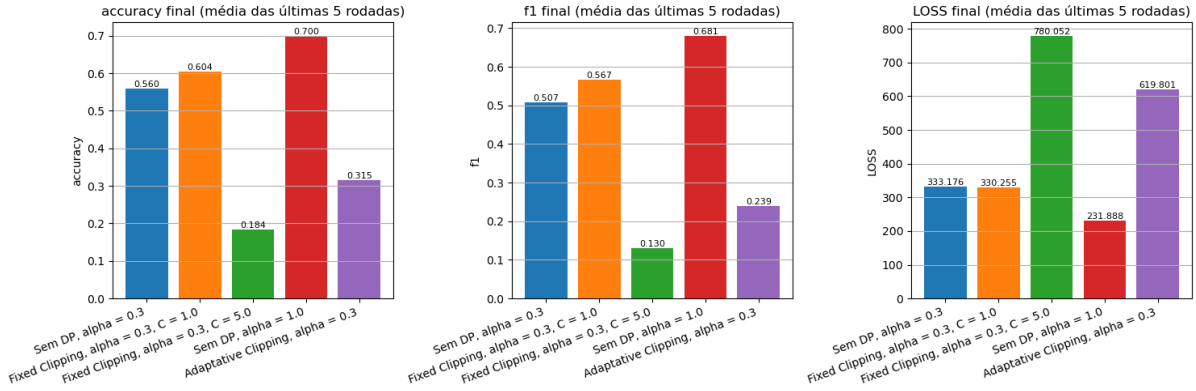


Figura 4: Métricas finais das técnicas de DP

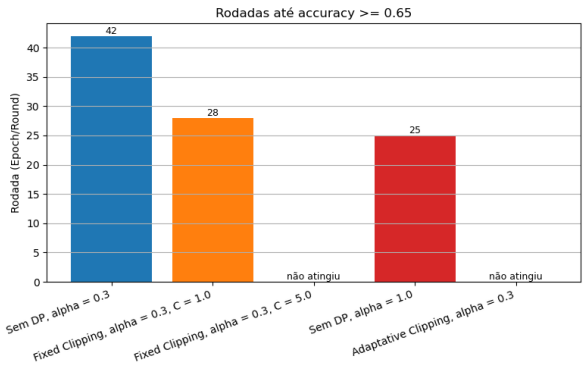


Figura 5: Rodadas necessárias até uma acurácia ≥ 0.65

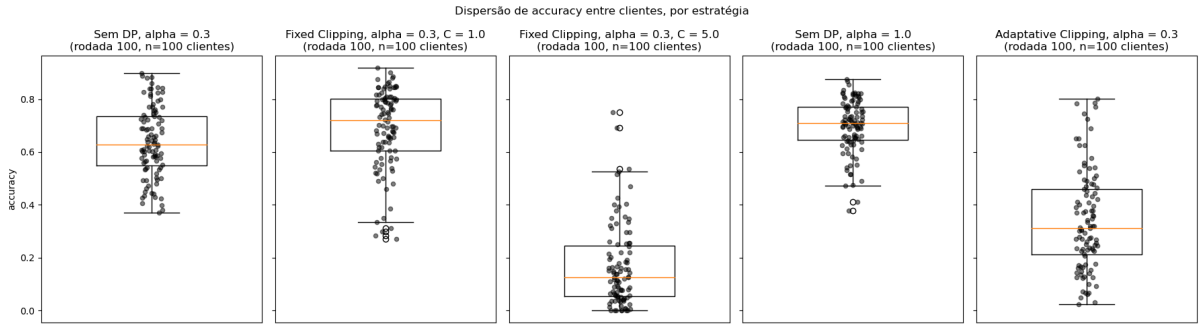


Figura 6: Dispersão da acurácia dos clientes

9 Discussão dos resultados

Como é possível observar na figura 2, todas as versões com $\alpha = 0.3$ demoraram mais para convergir do que a versão com $\alpha = 1.0$, evidenciando que é de fato mais difícil para os modelos convergirem em métricas boas em dados não-IID. Além disso, é visível que o Gradient Clipping de Corte Fixo com norma baixa, $C = 1.0$, desempenhou de forma similar ou até mesmo melhor que o *baseline* de $\alpha = 0.3$, em certas ocasiões, como evidenciado pela figura 3, que mostra que ambas as versões se manteram em uma tendência oscilatória, mas, próxima aos 0.6 e 0.7 de acurácia.

Evidenciado também pelas figuras 2 e 3, é possível ver que, tanto no caso do corte fixo, uma clipping norm C maior impactou negativamente as métricas do modelo, quanto, na versão sem DP, a simples má distribuição dos dados teve efeito similar, uma vez que o exato mesmo algoritmo obteve um desempenho, no geral pior no caso não-IID. Um resultado inesperado, à primeira vista é o de que no caso do clipping fixo com norma baixa, a qualidade do modelo, no geral, subiu com a adição de ruído, o que poderia ser explicado pela redução do overfitting nos dados de treino. Essa tendência pode ser observada tanto nas métricas finais dos modelos, em 4, quanto nas rodadas necessárias para se atingir uma acurácia ≥ 0.65 em 5, que foram menos para o caso com menos ruído e no caso IID, enquanto nos casos de alto ruído e de ruído adaptativo essa convergência sequer aconteceu.

Ao observar como a dispersão da acurácia ocorreu entre os clientes de cada método, é possível ver uma tendência esperada, mas, significativa. Nos casos com $\alpha = 0.3$, o ruído sempre aumentou a variância da distribuição, até mesmo no caso em que a média aumentou de Fixed Clipping $C = 1.0$, pois o ruído gaussiano acabou gerando alguns dados longe da média. Para o caso com clipping adaptativo, e distribuição IID, a dispersão variou menos que o baseline não-IID, o que, também, era de se esperar, dados os resultados anteriores.

9.1 Privacidade vs Utilidade

Ao analisarmos a acurácia dos modelos, observamos que existe um inerente *trade-off* entre privacidade e utilidade, e, calibrar esse tradeoff se torna mais difícil em distribuições de dados heterogêneas. A aplicação da Privacidade Diferencial introduziu distorções propositais no processo de otimização estocástica, e, embora essas medidas garantam uma delimitação contra o vazamento de informações contidas nos dados de treinamento originais de cada cliente, elas simultaneamente ofuscam o sinal verdadeiro do gradiente que a rede neural deveria seguir.

Porém, foi possível observar que, para casos realistas, ou seja, não-IID, um ruído pequeno pode simultaneamente melhorar a privacidade e as métricas do modelo, pois ele deixa de apenas memorizar os dados de treinamento, e acaba, por retornar respostas, na média, mais úteis, mesmo que com uma variância maior. Isso pode ser observado ao comparar as diferentes normas do Gradient Clipping fixo, na figura 7 ao lado.

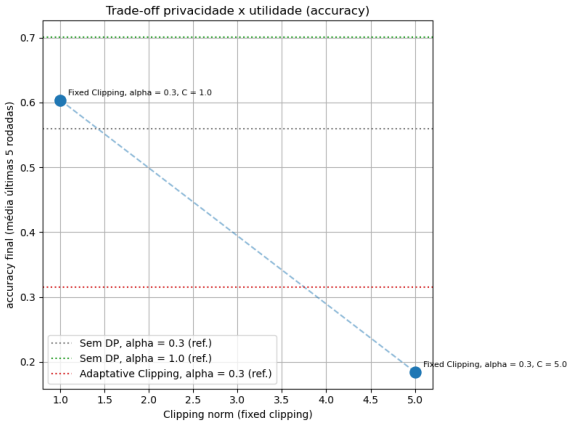


Figura 7: Trade-off entre privacidade e utilidade nos testes.

10 Conclusão

Assim, concluímos que a implementação de Privacidade Diferencial em sistemas de Aprendizado Federado sob condições de dados não-IID apresenta desafios técnicos significativos, mas também revela dinâmicas contraintuitivas e benéficas. Como era esperado, a heterogeneidade na distribuição dos dados, simulada com $\alpha = 0.3$ naturalmente dificultou a convergência dos modelos quando comparada a cenários IID ideais. A introdução de PD insere uma camada extra de complexidade, evidenciando um claro trade-off entre privacidade e utilidade, pois limites de corte mal calibrados ou agressivos, como o Fixed Clipping de $C = 5.0$ e estratégias adaptativas que têm dificuldades em lidar com a alta variância dos dados descentralizados acabam por introduzir um excesso de ruído que degrada severamente o desempenho global do modelo.

No entanto, nossos experimentos demonstraram um fenômeno importante. A injeção calibrada de ruído, aliada a um limite de corte adequado, como no caso do Fixed Clipping com $C = 1.0$, pode atuar como um poderoso mecanismo de regularização. Em vez de simplesmente prejudicar o aprendizado, a pequena ofuscação do gradiente mitigou o overfitting sobre os dados enviesados locais dos clientes. Isso resultou em métricas de acurácia e F1 Score mais estáveis e, em alguns momentos, superiores ao próprio baseline sem privacidade diferencial. Por fim, a utilização conjunta do framework Flower e da plataforma Flautim provou-se altamente eficaz para a orquestração e validação de cenários complexos e realistas. Os resultados reforçam a premissa de que não existe uma solução única para o treinamento federado diferencialmente privado, pois é possível atingir resultados similares de maneiras diferentes.

Referências

Yae Jee Cho, Jianyu Wang, and Gauri Joshi. Client selection in federated learning: Convergence analysis and power-of-choice selection strategies. *arXiv preprint arXiv:2010.01243*, 2020.

Flower Labs. Flower: A friendly federated learning framework, 2024. Acessado em: 21 jun. 2026.

FutureLab-DCC. Flautim tutoriais: Repositório oficial, 2024.

FutureLab-DCC. Tutorial 4 do Flautim: Jupyter Notebook, 2024.

FutureLab-DCC and Fundep. Flautim: Plataforma de aprendizado federado, 2024. Acessado em: 21 jun. 2026. Inclui as seções 'Sobre' e 'Como Funciona'.

Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.